

2. 최종 제안 모델: ECHO-HAMLET

최종 모델은 다음과 같이 정의하는 것이 가장 타당.

ECHO-HAMLET은 HAMLET의 단기 moment memory에 action-effect-aware episodic memory를 결합하고, 각 사건이 다양한 과거 기억 조합에서 미래 행동 손실을 얼마나 감소시키는지를 coalitional counterfactual supervision으로 측정하여 제한된 장기 메모리를 학습하는 VLA 정책.

전체 흐름은 다음과 같다.

1. HAMLET이 최근 관측의 단기 temporal context를 생성한다.
2. 현재 context를 query로 사용하여 장기 event memory를 검색한다.
3. 검색된 정보와 short-memory feature를 gated fusion한다.
4. 융합된 정보로 action chunk를 예측한다.
5. 행동 실행 후 관측된 상태 변화를 action-effect event로 표현한다.
6. 학습된 utility와 novelty를 이용하여 저장 여부를 결정한다.
7. 유사 사건은 병합하고, 메모리가 가득 차면 utility·recency·diversity가 낮은 slot을 제거한다.

기존 HAMLET은 최근 moment token을 효율적으로 처리하지만, 중요한 사건을 명시적으로 선택하여 장기간 보존하는 구조는 제한적

3. 모델 수식

3.1 현재 관측과 단기 기억

현재 관측 o_t 와 언어 명령 ℓ 을 VLM에 입력.

$$[h_t, m_t] = F_\theta(o_t, \ell)$$

여기서 h_t 는 action expert에 전달되는 시각·언어 feature이고, m_t 는 HAMLET moment representation.

$$\tilde{m}_t = H_\phi(m_{t-T_s+1:t})$$

$\tilde{m}_t$ 는 최근 T_s 개 시점으로부터 계산된 short-horizon context.

3.2 장기 event memory

시점 $t - 1$ 의 메모리를 다음과 같이 정의.

$$\mathcal{M}_{t-1} = \{(k_j, v_j, \bar{u}_j, \tau_j^{\text{last}}, n_j)\}_{j=1}^B$$

- k_j : 검색용 key
- v_j : action-effect event value
- $\bar{u}_j$: 누적 utility
- τ_j^{last} : 마지막 접근 또는 병합 시점
- n_j : effective merge count
- B : 최대 memory budget

3.3 Utility-aware memory retrieval

$$q_t = W_q[\tilde{m}_t; h_t; s_t; \ell]$$

$$c_{tj} = \frac{q_t^\top k_j}{\sqrt{d}} + b_\Delta(t - \tau_j^{\text{last}}) + \lambda_u \bar{u}_j$$

$$\mathcal{I}_t = \text{TopK}_j(c_{tj}, K)$$

$$r_t = \sum_{j \in \mathcal{I}_t} \text{softmax}(c_{tj}) v_j$$

이는 현재 상황과 관련이 있으면서, 과거에 실제 행동 예측에 유용했던 사건을 우선적으로 검색.

3.4 Gated fusion

$$g_t = \sigma(f_g([\tilde{m}_t; r_t; h_t]))$$

Warm-up 중에는 다음 scheduled gate floor를 사용.

$$\tilde{g}_t = \epsilon_e + (1 - \epsilon_e)g_t$$

$$z_t = \text{LN}(\tilde{m}_t + \tilde{g}_t \odot W_r r_t)$$

3.5 행동 예측

$$\hat{a}_{t:t+K-1} = \pi_\psi(h_t, z_t, s_t)$$

예측한 action chunk 중 현재 실행할 action을 $\bar{a}_t$ 라고 표기. 행동 이후 o_{t+1}, s_{t+1} 을 관측 .

3.6 Action-effect event encoding

$$\delta_t^v = \phi_\Delta([m_t; m_{t+1}; m_{t+1} - m_t])$$

$$\delta_t^s = W_s(s_{t+1} - s_t)$$

$$e_t = E_\eta([\tilde{m}_t; \bar{a}_t; s_t; \delta_t^v; \delta_t^s; \ell])$$

$$k_t = \text{normalize}(W_k e_t), \quad v_t = W_v e_t$$

따라서 메모리는 단순한 과거 영상이 아니라 다음 내용을 저장.

어떤 상황에서 어떤 행동을 수행했고, 그 결과 환경과 로봇 상태가 어떻게 변했는가?

3.7 Transition anti-collapse 학습

Action-effect prediction을 다음과 같이 추가.

$$\hat{\delta}_t = D_\zeta(m_t, \bar{a}_t, s_t)$$

$$\mathcal{L}_{\text{eff}} = \left\| \hat{\delta}_t - [\delta_t^v; \delta_t^s] \right\|_1$$

Transition embedding batch를 $D \in \mathbb{R}^{N \times d}$ 라고 하면,

$$\mathcal{L}_{\text{var}} = \frac{1}{d} \sum_q \max \left(0, \gamma - \sqrt{\text{Var}(D_{:q})} + \varepsilon \right)$$

$$\mathcal{L}_{\text{cov}} = \frac{1}{d} \sum_{p \neq q} \text{Cov}(D)_{pq}^2$$

이는 VICReg의 variance-covariance 원리를 참고한 anti-collapse regularization. 다만 본 모델에서는 동일 영상의 두 augmentation을 맞추는 것이 아니라 action-effect prediction을 의미적 supervision으로 사용한다는 점을 구분.

4. Coalitional Value of Memory

기존 단일 사건 제거 방식은 서로 중복되는 두 기억이 있을 때 각 기억의 가치를 지나치게 낮게 평가할 수 있다. 최종 모델에서는 다양한 기억 조합에서 해당 사건의 marginal contribution을 측.

Memory subset S 를 사용했을 때 미래 행동 손실을 다음과 같이 정의

$$J_t(S) = \frac{1}{H_u} \sum_{\tau=t+1}^{t+H_u} \mathcal{L}_{\text{act}}^{\text{EMA}}(\tau \mid \mathcal{M}_S)$$

사건 e_t 의 coalitional utility target은

$$u_t^* = \mathbb{E}_{S \sim p(\mathcal{M} \setminus \{e_t\})} [J_t(S) - J_t(S \cup \{e_t\})]_+$$

. 즉, 사건 e_t 를 coalition에 추가했을 때 미래 행동 손실이 감소한 정도를 측정. 전체 조합을 계산하면 비용이 크므로 candidate event마다 소수의 coalition을 Monte Carlo sampling. 이 계산은 완성된 trajectory와 미래 action target을 요구하므로 학습 시에만 수행. 온라인 utility predictor는 다음과 같다.

$$\hat{u}_t = U_\xi(e_t, \tilde{m}_t, r_t)$$

$$\mathcal{L}_u = \text{Huber}(\hat{u}_t, \text{sg}(\text{Norm}(u_t^*)))$$

여기서 sg는 stop-gradient.

5. 선택적 저장과 메모리 관리

Novelty는 다음과 같이 정의.

$$\nu_t = 1 - \max_j \cos(k_t, k_j)$$

저장 확률은

$$\omega_t = \sigma(f_w(e_t, \nu_t, \hat{u}_t))$$

로 계산. CVoM으로부터 얻은 binary target은

$$y_t^* = 1[u_t^* > \delta_u]$$

이며,

$$\mathcal{L}_{\text{write}} = \text{BCE}(\omega_t, y_t^*)$$

로 write gate를 학습.

과도한 저장을 막기 위해 다음 budget loss를 사용.

$$\mathcal{L}_{\text{budget}} = \left(\frac{1}{T} \sum_t \omega_t - \rho \right)^2$$

유사 event 병합

$$j^* = \arg \max_j \cos(k_t, k_j)$$

유사도가 δ_{merge} 보다 크면 새로운 slot을 만들지 않고 병합.

$$\eta_t = \frac{\omega_t}{n_{j^*} + \omega_t}$$

$$k_{j^*} \leftarrow \text{normalize}((1 - \eta_t)k_{j^*} + \eta_t k_t)$$

$$v_{j^*} \leftarrow (1 - \eta_t)v_{j^*} + \eta_t v_t$$

$$\bar{u}_{j^*} \leftarrow (1 - \eta_u)\bar{u}_{j^*} + \eta_u \hat{u}_t$$

$$\tau_{j^*}^{\text{last}} \leftarrow t, \quad n_{j^*} \leftarrow n_{j^*} + \omega_t$$

업로드된 수정안의 key, utility, timestamp 갱신뿐 아니라 value와 effective count까지 갱신해야 slot 내부 상태가 일관.

Memory eviction

메모리가 가득 찬 경우 각 slot의 retention score를 계산.

$$R_j = \lambda_1 \bar{u}_j + \lambda_2 \exp\left(-\frac{t - \tau_j^{\text{last}}}{\kappa}\right) + \lambda_3 \left(1 - \max_{l \neq j} \cos(k_j, k_l)\right)$$

$$j_{\text{evict}} = \arg \min_j R_j$$

이에 따라 utility가 낮고 오래되었으면 다른 slot과 중복되는 기억을 먼저 제거.

6. Algorithm 1: Online inference

Input:

observation o_t , robot state s_t , instruction ℓ ,
short-memory state, long-term memory $M_{-}(t-1)$

1. $[h_t, m_t] \leftarrow \text{VLM}(o_t, \ell)$
2. $\tilde{m}_t \leftarrow \text{HAMLET-ShortMemory}(m_{-}(t-T_s+1:t))$
3. $q_t \leftarrow W_q[\tilde{m}_t; h_t; s_t; \ell]$
4. $I_t \leftarrow \text{UtilityAwareTopK}(q_t, M_{-}(t-1))$
5. $r_t \leftarrow \text{Attend}(q_t, M_{-}(t-1)[I_t])$
6. $g_t \leftarrow \text{sigmoid}(f_g([\tilde{m}_t; r_t; h_t]))$
7. $z_t \leftarrow \text{LN}(\tilde{m}_t + g_t \odot W_r r_t)$
8. $\hat{a}_{-}(t:t+K-1) \leftarrow \text{ActionExpert}(h_t, z_t, s_t)$
9. Execute current action $\hat{a}_t$
10. Observe $o_{-}(t+1), s_{-}(t+1)$
11. Construct action-effect event e_t
12. $k_t \leftarrow \text{normalize}(W_k e_t), v_t \leftarrow W_v e_t$
13. $\hat{u}_t \leftarrow \text{UtilityPredictor}(e_t, \tilde{m}_t, r_t)$

```

14.  $v_t \leftarrow 1 - \max_j \text{cosine}(k_t, k_j)$ 
15.  $\omega_t \leftarrow \text{WriteGate}(e_t, v_t, \hat{u}_t)$ 

16. if  $\omega_t > \delta_w$ :
    if max similarity  $> \delta_{\text{merge}}$ :
        merge key, value, utility, timestamp and count
    else:
        allocate a new slot
        if memory is full:
            evict slot with minimum retention score

17. Return updated memory  $M_t$ 

```

온라인 알고리즘에는 CVoM teacher, 미래 action loss, Monte Carlo coalition sampling 및 warm-up gate floor가 포함되지 않는다.

7. Algorithm 2: Offline training

```

Stage A – Action-effect bootstrap
1. Encode transition events from complete trajectories.
2. Train the transition encoder using:
     $\mathcal{L}_{\text{eff}} + \lambda_{\text{var}} \mathcal{L}_{\text{var}} + \lambda_{\text{cov}} \mathcal{L}_{\text{cov}}$ 
3. Bootstrap memory using novel or demonstrably useful events.

Stage B – Retrieval and fusion warm-up
4. Train retrieval, fusion and action policy.
5. Use the differentiable gate floor:
     $\tilde{g}_t = \varepsilon_e + (1 - \varepsilon_e)g_t$ 
6. Gradually decay  $\varepsilon_e$  to zero.

Stage C – Coalitional CVoM supervision
7. Freeze an EMA snapshot of the policy and memory encoder.
8. Sample candidate events and memory coalitions.
9. Compute future action loss  $J_t(S)$  and  $J_t(S \cup \{e_t\})$ .
10. Estimate the MC coalitional target  $u_t^*$ .
11. Train the utility predictor and write gate.
12. Jointly fine-tune the policy, retrieval and memory modules.

```

전체 목적함수는 다음과 같이 구성할 수 있다.

$$\mathcal{L} = \mathcal{L}_{\text{act}} + \lambda_{\text{eff}} \mathcal{L}_{\text{eff}} + \lambda_{\text{var}} \mathcal{L}_{\text{var}} + \lambda_{\text{cov}} \mathcal{L}_{\text{cov}} + \lambda_u \mathcal{L}_u + \lambda_w \mathcal{L}_{\text{write}} + \lambda_b \mathcal{L}_{\text{budget}}$$

9. 참신성의 핵심

MemoryVLA는 perceptual–cognitive memory bank와 memory consolidation을 사용하고, ReMem-VLA는 frame/chunk 수준 recurrent memory를, LaMem-VLA는 short/long latent memory vault를 제안. MemoryVLA , ReMem-VLA , LaMem-VLA

따라서 단순한 장·단기 메모리 분리만으로는 충분한 novelty가 되기 어렵다. 최종 ECHO-HAMLET의 차별점은 다음 세 요소의 결합에 있다.

- 관측 frame이 아닌 action consequence를 event로 저장
- 단일 삭제가 아닌 coalition 기반 미래 행동 기여도 측정
- 계산량이 큰 counterfactual utility를 경량 online write policy로 distillation

Gate floor와 variance/covariance loss는 중요한 학습 안정화 요소이지만, 이를 논문의 주요 novelty로 주장해서는 안 된다. 논문의 핵심 기여는 **coalitional action-effect memory consolidation**으로 설정하는 것이 적절.

CVPR 수준의 방법론적 설계로는 발전했지만, 채택 가능성은 다음 비교·ablation을 얼마나 엄밀하게 수행하느냐에 달려 있다.

- HAMLET $T = 4, 8, 16$
- 단순 FIFO 및 frame memory
- MemoryVLA, ReMem-VLA, LaMem-VLA
- single-removal CVoM과 coalition CVoM
- hard floor, scheduled floor, floor 없음
- effect loss 및 variance/covariance 제거

- bootstrap 제거
- merge field별 ablation
- memory budget $B = 8, 16, 32, 64$
- 긴 가림, 반복 횟수 기억, subtask 순서, 실패 후 복구
- 성공률, write rate, slot churn, 잘못된 retrieval, latency와 peak memory
- 최소 3~5개 random seed의 평균과 표준편차

특히 coalition sampling의 학습 비용과 샘플 수에 따른 성능·비용 trade-off를 반드시 보고해야 .
함